User-Agent Header

Description	Standard request header to helps about antibot

What is?

It identifies the request sender's device which includes information about the device type, operating system, browser name and version.

The value of the user agent header is called **user agent string**. It follows a straightforward format that provides various details about the connecting user:

```
User-Agent: Mozilla/5.0 (<system-information>) <platform> <br
owser-info>
```

- `Mozilla/5.0` Historical artifact
- `<system-information>` Represents the operating system, its version and the CPU architecture.
- `<platform>` Represent the rendering engine used and its version.
- `<browser-info>` Provides information about the used web browser and it's version.

```
Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Geck
o/20100101 Firefox/113.0
```

Not all user agent strings are of the same value. For example:

- Newer browser versions are more likely to be successful.
- Windows and MacOs operating systems are more likely to be successful.
- More popular web browsers like Chrome and Firefox are more likely to succeed.

Why do we need to add user-agent headers for our code?

The User-Agent header is used to add metadata to the request. Websites lookup this metadata to determine if the request sender is a human user or a bot. Therefore, adding user agents to web scrapers helps avoid web scraping blocking.

`ua_parser`

Is an open-source library used for parsing User-Agent strings, which are text strings sent by web browsers and other applications to identify themselves on the Internet.
In essence helps interpret the information contained in User-Agent strings

```
from ua_parser import user_agent_parser
```

We can use this library to obtain information about the User-Agent strings

```
from ua_parser import user_agent_parser

user_agent_string = "Mozilla/5.0 (Windows NT 10.0; Win64; x6
4) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/58.0.3029.11
0 Safari/537.3"
parsed_user_agent = user_agent_parser.Parse(user_agent_strin
g)

print(parsed_user_agent)
```

The result will contain information about the browser, version, operating system, and other details extracted from the User-Agent string.

User-Agent scraping

The user-agent are vital for scraping why the websites often analyze User-Agent headers to determine if the request sender is a real user or a bot.

Many websites are likely to block requests that don't provide *any* User-Agent headers. While most throttle or block specific User-Agent headers that appear unusual like Linux operating system or unknown web browser.

Datasets of user-agent strings

We need to rotate the dataset and semi-randomly for the best scraping results.

To scrape user agent strings from useragentlist.net online database we'll use httpx to retrieve the HTML page and `bs4` to parse the HTML and extract the user agents:

```
pip install httpx beautifulsoup4 ua-parser
```

```
from bs4 import BeautifulSoup
import httpx

url = "https://www.useragentlist.net/"
request = httpx.get(url)
user_agents = []
soup = BeautifulSoup(request.text, "html.parser")
for user_agent in soup.select("pre.wp-block-code"):
    user_agents.append(user_agent.text)
```

Some of the user-agent we get:

```
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/1
13.0.0.0 Safari/537.36 Edg/113.0.1774.35
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Ch
rome/113.0.0.0 Safari/537.36 Edg/113.0.1774.35
Mozilla/5.0 (Windows NT 6.3; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/11
3.0.0.0 Safari/537.36 Edg/113.0.1774.35
Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/11
3.0.0.0 Safari/537.36 Edg/113.0.1774.35
Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/11
3.0.0.0 Safari/537.36 Edg/113.0.1774.35
...
```

```
...
...
```

Rotate the user-agent headers

> 📌 The idea behind rotating user agents is to change the user agent for every request being made by the scraper. This will **prevent user-agent based blocking** as the traffic splits between all user agents.

We could just grab a random user agent each time

The rotation criteria will be defined based on weights, where user agents with higher weights will get higher priority.

The weighting criteria is defined based on several factors

- Device
- Operating stystem
- Web browser

For example, user agents for Mac and Windows devices will get higher weights than Linux and Android devices.

```
from ua_parser import user_agent_parser
from functools import cached_property
from typing import List
from time import time
import random

class UserAgent:
    """container for a User-Agent"""

    def __init__(self, string) -> None:
```

```
        self.string: str = string
        # Parse the User-Agent string
        self.parsed_string: str = user_agent_parser.Parse(str
ing)
        self.last_used: int = time()

    # Get the browser name
    @cached_property
    def browser(self) -> str:
        return self.parsed_string["user_agent"]["family"]

    # Get the browser version
    @cached_property
    def browser_version(self) -> int:
        return int(self.parsed_string["user_agent"]["major"])

    # Get the operation system
    @cached_property
    def os(self) -> str:
        return self.parsed_string["os"]["family"]

    # Return the actual user agent string
    def __str__(self) -> str:
        return self.string

class Rotator:
    """weighted random user agent rotator"""

    def __init__(self, user_agents: List[UserAgent]):
        # Add User-Agent strings to the UserAgent container
        user_agents = [UserAgent(ua) for ua in user_agents]
        self.user_agents = user_agents

    # Add weight for each User-Agent
    def weigh_user_agent(self, user_agent: UserAgent):
        weight = 1_000
```

```
        # Add higher weight for less used User-Agents
        if user_agent.last_used:
            _seconds_since_last_use = time() - user_agent.las
t_used
            weight += _seconds_since_last_use
        # Add higher weight based on the browser
        if user_agent.browser == "Chrome":
            weight += 100
        if user_agent.browser == "Firefox" or "Edge":
            weight += 50
        if user_agent.browser == "Chrome Mobile" or "Firefox
Mobile":
            weight += 0
        # Add higher weight for higher browser versions
        if user_agent.browser_version:
            weight += user_agent.browser_version * 10
        # Add higher weight based on the OS type
        if user_agent.os == "Windows":
            weight += 150
        if user_agent.os == "Mac OS X":
            weight += 100
        if user_agent.os == "Linux" or "Ubuntu":
            weight -= 50
        if user_agent.os == "Android":
            weight -= 100
        return weight

    def get(self):
        # Weigh all User-Agents
        user_agent_weights = [
            self.weigh_user_agent(user_agent) for user_agent
in self.user_agents
        ]
        # Select a random User-Agent
        user_agent = random.choices(
            self.user_agents,
```

```
            weights=user_agent_weights,
            k=1,
        )[0]
        # Update the last used time when selecting a User-Age
nt
        user_agent.last_used = time()
        return user_agent
```

First, the User-Agent strings are passed to the `UserAgent` class to extract its properties using the `ua_parser` module. Then, we create `Rotator` object which can select a random `UserAgent` from the pool based on the weights we defined.